# Problem Statement

## Database Monitoring & Alerting Issues

---

## Executive Summary

Our database monitoring system is experiencing critical alerting deficiencies that prevent effective incident detection, routing, and remediation across multiple database environments and geographic regions. This is causing delayed response times, missed SLA breaches, and operational inefficiencies.

---

## Problem 1: Single Alert for Multiple Database Failures

### Current Situation

When multiple databases experience failures across different regions and environments simultaneously, the monitoring system generates only ONE consolidated alert ticket instead of creating individual tickets for each unique database failure.

### Real Example - October 2, 2025

**Multiple Database Failures Detected:**

| Timestamp | Region | Environment | Database Name | Failure Type | Incident ID |
|---|---|---|---|---|---|
| 10/2/2025, 11:17:11 AM | EU-WEST | Production | customer-orders-db | Connection timeout | db-conn-8a5f-4e2a |
| 10/2/2025, 11:05:43 AM | US-EAST | Production | payment-transactions-db | Deadlock detected | db-dead-6c1-de5a |
| 10/2/2025, 11:52:26 AM | APAC-SG | Production | inventory-master-db | Disk space 95% full | db-disk-560-8b3c |
| 10/2/2025, 1:45:50 AM | EU-CENTRAL | Production | user-profile-db | Replication lag 30 min | db-repl-268-8339 |
| 10/2/2025, 11:22:32 AM | EU-WEST | UAT | order-analytics-db | Query timeout | db-qry-096d-9168 |
| 10/2/2025, 1:26:06 AM | US-WEST | Production | product-catalog-db | High CPU 98% | db-cpu-2a28-9c94 |

**System Response:**

- Created: **1 single ServiceNow ticket** titled "Database Performance Issues"
- Ticket contained: Generic message "Multiple database alerts triggered"

- Assignment: Sent to generic "Database Support" queue
- Context missing: Which databases? Which regions? What failure types?

## What We Expected

Each database failure should create its own ticket:

- **Ticket 1**: Customer Orders DB (EU-WEST) - Connection timeout → UK Database Team
- **Ticket 2**: Payment Transactions DB (US-EAST) - Deadlock → US Database Team
- **Ticket 3**: Inventory Master DB (APAC-SG) - Disk space → Singapore Database Team
- **Ticket 4**: User Profile DB (EU-CENTRAL) - Replication lag → Germany Database Team
- **Ticket 5**: Order Analytics DB (EU-WEST UAT) - Query timeout → UAT Support Team
- **Ticket 6**: Product Catalog DB (US-WEST) - High CPU → US Database Team

## Business Impact

**Operational Impact:**

- **Response Time Delayed**: Team sees one generic ticket, doesn't realize 6 critical databases are down
- **Wrong Team Assignment**: Generic ticket goes to central queue instead of regional experts
- **Missing Context**: Engineers don't know severity, affected region, or failure type without investigation
- **Lost Visibility**: Cannot track individual database incidents separately
- **SLA Tracking Impossible**: Cannot measure response time per database or region

**Customer Impact:**

- Customer Orders DB down → Customers cannot place new orders (Revenue loss)
- Payment Transactions DB deadlocked → Payment processing failures (Business critical)
- Inventory Master DB full → Stock updates failing (Operational impact)
- User Profile DB replication lag → Inconsistent user data across regions
- Product Catalog DB high CPU → Slow product searches and browsing

**Team Confusion:**

- UK DBA team doesn't know customer-orders-db needs immediate attention
- US team unaware of payment-transactions-db deadlock
- Singapore team not alerted about disk space emergency
- Germany team missing replication lag issue
- Multiple teams waiting for someone else to take action

### The Core Problem

**Current Behavior:**

```
6 Different Database Failures
  ↓
Monitoring System Aggregates All
  ↓
1 Combined Alert Generated
  ↓
1 Generic ServiceNow Ticket Created
  ↓
Missing: Region, Environment, Database Name, Failure Type, Severity
```

**Expected Behavior:**

```
6 Different Database Failures
  ↓
6 Individual Alerts Generated
  ↓
6 Separate ServiceNow Tickets Created
  ↓
Each Ticket Contains: Complete context (Region, Env, DB Name, Failure Type, Metrics, Assignment)
```

---

## Problem 2: Email Alerts Firing on Every Health Check

### Current Situation

Database health check monitoring sends email alerts on EVERY execution, regardless of whether databases are healthy or experiencing issues. This results in hundreds of unnecessary emails daily, causing severe alert fatigue.

### Real Example - Daily Email Volume

**October 2, 2025 - Email Statistics:**

- Total health check emails received: **576 emails**
- Actual database failures: **6 incidents**
- False positive emails (healthy checks): **570 emails**
- False positive rate: **98.9%**

### What's Happening

**Current Email Behavior:**

```
09:00 AM - Health Check Runs → Email Sent: "All 48 databases healthy ✓"
09:15 AM - Health Check Runs → Email Sent: "All 48 databases healthy ✓"
09:30 AM - Health Check Runs → Email Sent: "All 48 databases healthy ✓"
09:45 AM - Health Check Runs → Email Sent: "All 48 databases healthy ✓"
10:00 AM - Health Check Runs → Email Sent: "All 48 databases healthy ✓"
... (Continues every 15 minutes, 24/7)
```

**Every 15 minutes:**

- Health check script executes for all 48 production databases
- Script completes checking connection, performance, disk space, replication
- Email notification automatically triggers
- Email sent to: 15 database engineers + 3 manager distribution lists
- Email contains: Empty results or "All systems healthy" message

**Daily Volume Per Person:**

- Emails per hour: 4 health checks/hour × 48 databases = 192 emails/hour
- Emails per day: 192 × 24 hours = 4,608 potential emails/day
- Actual emails: Filtered to summaries, but still 30-50 unnecessary emails/day per person

## Examples of Unnecessary Emails

**Email 1 - Empty Results:**

```
Subject: Database Health Check - customer-orders-db
Time: 10/2/2025 09:15 AM

Status: Running
Results: [Empty]
No issues detected
```

**Email 2 - Successful Check:**

Subject: Database Health Check - payment-transactions-db
Time: 10/2/2025 09:30 AM

Status: Completed Successfully
Connection: OK
Performance: Within SLA
Disk Space: 67% used
Replication: 0 lag

Action Required: None

**Email 3 - Redundant Success:**

Subject: Scheduled Database Monitoring - inventory-master-db
Time: 10/2/2025 09:45 AM

Health Check Executed: Yes
Issues Found: No
Details: All metrics normal

## What We Expected

Emails should ONLY be sent when there's an actual problem:

**Should Send Email:**

- Connection timeout detected → Email: "ALERT: customer-orders-db connection failure"
- Disk space > 90% → Email: "WARNING: inventory-master-db disk space critical"
- Replication lag > 15 minutes → Email: "ALERT: user-profile-db replication behind"
- Deadlock detected → Email: "CRITICAL: payment-transactions-db deadlock"
- CPU > 90% for 10 minutes → Email: "WARNING: product-catalog-db high CPU"

**Should NOT Send Email:**

- All checks pass successfully
- Databases performing within SLA
- No issues detected during health check
- Routine monitoring completing normally

## Business Impact

**Alert Fatigue:**

- Engineers receive 30-50 database emails daily

- Real critical alerts buried in flood of "all healthy" emails
- Team stops reading database alert emails altogether
- Genuine failures get missed or delayed response

**Time Waste:**

- Engineers spend 15-20 minutes each morning filtering emails
- Must manually review each email to determine if action needed
- Team estimates 2 hours/day wasted across 15 engineers = 30 hours/day lost

**Missed Critical Alerts:**

- October 1: Payment DB deadlock alert missed for 45 minutes (buried in healthy check emails)
- September 28: Disk space alert missed, database went read-only before detection
- September 25: Replication failure not noticed for 2 hours

**Email System Impact:**

- Database monitoring generating 60,000+ emails per week
- Email server flagging monitoring as potential spam
- Legitimate emails sometimes delayed or filtered

## Root Cause

**The Problem:** The database health check monitoring script was configured with a workaround solution using "sendMail" function. This function was added to ensure notifications are sent, BUT it lacks conditional logic.

**How It Works Now:**

```
Step 1: Health check script runs (every 15 minutes)
Step 2: Script checks database health
Step 3: Script completes
Step 4: sendMail function executes (ALWAYS - no condition check)
Step 5: Email sent (regardless of results)
```

**Missing Conditional Logic:**

- Script does NOT check "if failure detected THEN send email"
- Script does NOT check "if metrics exceed threshold THEN send email"
- Script does NOT check "if status = healthy THEN skip email"
- sendMail function has no IF/THEN condition capability

**Why Workaround Was Used:**

- Original monitoring service broker task unable to generate email alerts
- Engineering team needed quick solution to get notifications
- Added sendMail as temporary workaround
- Workaround became permanent, never properly fixed

### The Core Problem

**Current Behavior:**

```
Health Check Executes (Every 15 min)
  ↓
Checks 48 Databases
  ↓
Results: 47 Healthy, 1 Failed
  ↓
sendMail() Executes (No Condition)
  ↓
48 Emails Sent (All checks, healthy + failed)
  ↓
Engineers Overwhelmed
```

**Expected Behavior:**

```
Health Check Executes (Every 15 min)
  ↓
Checks 48 Databases
  ↓
Results: 47 Healthy, 1 Failed
  ↓
IF (Failure Detected) THEN sendMail()
  ↓
1 Email Sent (Only for failed database)
  ↓
Engineers Get Actionable Alert
```

---

## Impact Summary

### Operational Metrics (Current State)

| Metric | Current Performance | Target Performance | Gap |
|---|---|---|---|
| Alert-to-Ticket Ratio | 6 failures → 1 ticket | 6 failures → 6 tickets | 83% miss rate |
| Email False Positive Rate | 98.9% | < 5% | 93.9% excessive |

| Metric | Current Performance | Target Performance | Gap |
|---|---|---|---|
| Average Response Time | 45 minutes | 15 minutes | 200% slower |
| Missed Critical Alerts | 3 in last 7 days | 0 | Unacceptable |
| Engineer Time Waste | 30 hours/day team-wide | 0 hours/day | High cost |
| SLA Tracking Accuracy | Unable to track | 100% accurate | Complete failure |

### Customer Impact

- **Revenue Loss**: Payment processing delays during undetected failures
- **Service Degradation**: Slow response times when databases under stress
- **Data Inconsistency**: Replication lag causing incorrect information
- **Order Failures**: Unable to place orders when order DB down
- **Poor User Experience**: Slow page loads, timeouts, errors

### Team Impact

- **Burnout Risk**: Constant alert noise reducing team effectiveness
- **Reduced Confidence**: Team doesn't trust monitoring system
- **Manual Overhead**: Engineers manually checking database health
- **Delayed Response**: Critical issues hidden in noise
- **Morale Issues**: Frustration with broken alerting system

---

## Affected Systems

### Database Environments

- **Production Databases**: 48 databases across 4 regions
- **UAT/Staging**: 12 databases
- **Development**: 8 databases

### Geographic Regions

- **EU-WEST** (London, Dublin): 18 databases
- **EU-CENTRAL** (Frankfurt): 12 databases
- **US-EAST** (Virginia): 10 databases
- **US-WEST** (Oregon): 8 databases
- **APAC-SG** (Singapore): 6 databases
- **APAC-IN** (Mumbai): 4 databases

### Database Types Affected

- **Transactional Databases**: PostgreSQL, MySQL (Customer orders, payments, user profiles)
- **Analytics Databases**: Redshift, Snowflake (Reporting, business intelligence)
- **Caching Databases**: Redis, Memcached (Session management, application cache)
- **Document Databases**: MongoDB (Product catalog, content management)

### Critical Business Databases

- **customer-orders-db**: Order processing system (Revenue critical)
- **payment-transactions-db**: Payment processing (Business critical)
- **inventory-master-db**: Stock management (Operational critical)
- **user-profile-db**: Customer authentication and profiles
- **product-catalog-db**: E-commerce product information

---

## Key Questions Requiring Answers

### Technical Questions

1. Why is the monitoring system aggregating multiple failures into one alert?
2. Where in the alerting pipeline is the aggregation happening? (Prometheus? Alert Manager? ServiceNow webhook?)
3. Are alert labels configured to include region, environment, and database identifiers?
4. Why does sendMail function lack conditional logic?
5. Can the monitoring service broker support conditional alerting natively?
6. What is the current alert grouping configuration?
7. Are there ServiceNow business rules auto-merging similar incidents?

### Process Questions

8. Who owns the monitoring system configuration?
9. What is the approval process for monitoring system changes?
10. How long has this problem existed?
11. Have previous attempts been made to fix it? Why did they fail?
12. What is the acceptable timeline for permanent fix implementation?

### Business Questions

13. What is the estimated cost of delayed incident response?

14. How many SLA breaches have occurred due to missed alerts?
15. What is the business case for prioritizing this fix?
16. Are there compliance or audit requirements affected by inadequate alerting?

---

## Urgency & Priority

### Priority Level: P1 - CRITICAL

**Justification:**

- Revenue-impacting systems affected (payment processing)
- Multiple SLA breaches occurring
- Customer experience degraded
- Team effectiveness severely reduced
- Compliance risk (inability to track and respond to incidents)

### Required Action

**Immediate (Next 48 Hours):**

- Implement manual workarounds to create individual tickets
- Set up manual monitoring rotations to catch missed alerts
- Filter unnecessary emails to reduce alert fatigue

**Short-Term (Next 2 Weeks):**

- Root cause analysis of alert aggregation issue
- Design solution for conditional email alerting
- Prioritize permanent fix implementation

**Long-Term (Next 30 Days):**

- Implement permanent technical solution
- Validate alert accuracy and routing
- Restore team confidence in monitoring system
- Document lessons learned and prevent recurrence

---

## Next Steps

This problem statement will be used to:

1. **Create SRE Runbook** - Document workarounds and troubleshooting procedures
2. **Develop Incident Management Process** - Define escalation and response procedures
3. **Propose Technical Solutions** - Design permanent fixes for both problems
4. **Secure Resources** - Get engineering time allocated for implementation
5. **Track Progress** - Monitor improvement in alerting quality and team satisfaction

---

**Document Owner**: Database SRE Team Lead
**Date Created**: October 10, 2025
**Status**: Active Problem - Requires Urgent Resolution
**Review Date**: Weekly until resolved